程序计数器的作用，为什么是私有的?

程序计数器（Program Counter Register）的作用主要有两个：

第一，**记录当前线程下一条将要执行的字节码指令地址。**
在 JVM 中，线程执行的是字节码，每条指令执行完之后，都需要知道下一步从哪里继续执行，因此程序计数器就是用来保存当前执行位置的。

第二，**在线程切换时恢复执行现场。**
Java 是支持多线程的，CPU 采用时间片轮转的方式调度线程。当一个线程执行过程中被挂起，切换到其他线程时，需要保存当前执行状态。等该线程再次获得 CPU 时，需要从上次中断的位置继续执行，而程序计数器正是保存这个执行位置的关键数据。

✅ 为什么程序计数器是线程私有的?

因为每个线程的执行路径不同，所执行的字节码指令也不同。
如果多个线程共享同一个程序计数器，那么在线程切换时会互相覆盖执行位置，导致线程无法正确恢复执行，程序会发生混乱。

因此，为了保证线程切换后能够恢复到正确的执行位置，**每个线程都必须拥有独立的程序计数器，这也是它被设计为线程私有的根本原因。**

✅ 补充（加分点）

1. 程序计数器是 JVM 规范中唯一一个**不会发生 OutOfMemoryError 的区域。**
2. 如果线程正在执行 Java 方法，程序计数器记录的是字节码指令地址；如果执行的是 Native 方法，则程序计数器为空（Undefined）。

方法区中的方法的执行过程?

当程序中对象或类直接调用某个方法时
首先，JVM会根据方法的符号引用找到实际的方法地址
接着，JVM会在当前线程的Java虚拟机栈中为该方法分配一个新的栈帧，用于存储局部变量表、操作数栈、动态链接、方法出口等信息

然后，执行方法中的字节码指令，方法执行结束后，可能会返回一个结果给调用者，并清理当前栈帧，恢复调用者的执行环境

方法区中有哪些东西?

方法区用于存储已被虚拟机加载的类型信息、常量、静态变量、即时编译器编译后的代码缓存等

① 类的元数据（Class Metadata）

包括：

- 类的名称、修饰符
- 父类和接口信息
- 字段和方法的描述信息
- 访问控制信息

这些信息在类加载时生成，用于 JVM 运行时解析和调用。

---

② 运行时常量池（Runtime Constant Pool）

它是类文件常量池在运行时的表现形式，主要包括：

- 字面量（如字符串、基本类型常量）
- 符号引用（如类、字段、方法的符号引用）

在类加载后，这些符号引用会在运行时被解析为直接引用。

👉 运行时常量池是方法区中最重要的一部分。

---

③ 静态变量（Static Variables）

类的静态变量会随着类的加载而分配，并存储在方法区中。
这些变量在类初始化阶段被赋值，生命周期与类一致。

---

④ 方法信息和字节码

包括：

- 方法的参数、返回值类型
- 方法访问修饰符
- 方法的字节码（即编译后的指令）

JVM 通过这些字节码执行方法。

---

⑤ 即时编译器（JIT）生成的代码缓存

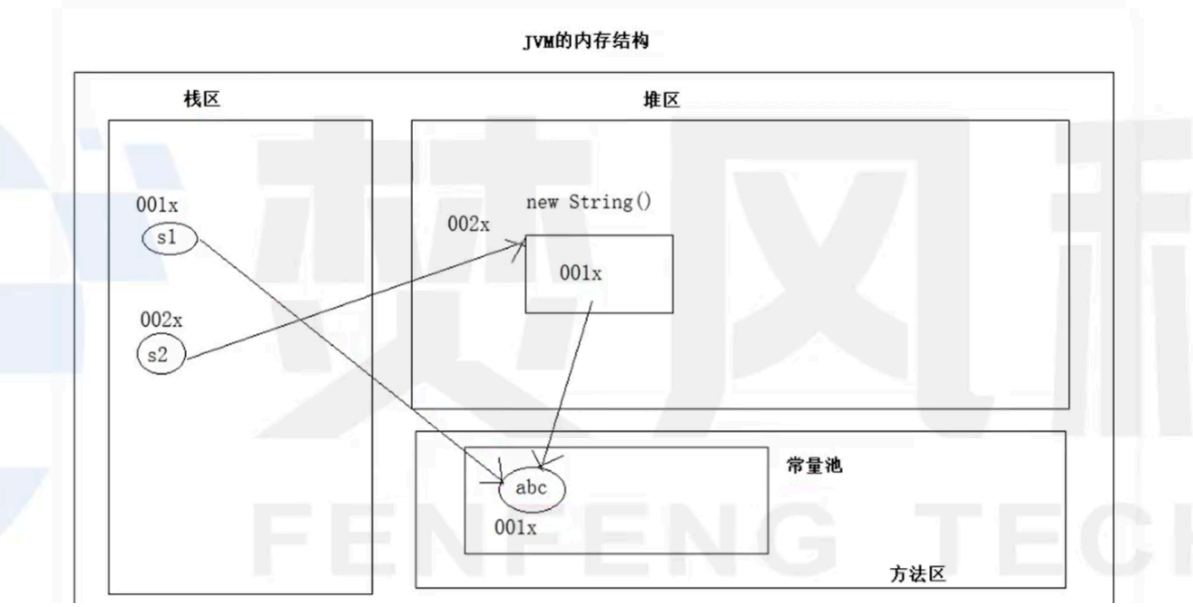
部分 JVM 实现（如 HotSpot）会将热点代码编译为机器码，并缓存起来，提高执行效率。

注意：
这部分属于具体 JVM 实现，不是 JVM 规范强制要求。

String保存在哪里?

String保存在字符串常量池，它的值是不可变的，可以被多个引用共享

String s1="abc"和String s2=new String("abc")的区别



✅ 面试回答版本

String s1 = "abc" 和 String s2 = new String("abc") 的主要区别在于对象创建方式和内存位置不同。

---

① 创建方式不同

```
String s1 = "abc";
```

这种方式是**直接使用字符串字面量**创建对象。
JVM 会先去**字符串常量池（String Constant Pool）**中查找是否已经存在 `"abc"`：

- 如果存在，就直接返回该对象的引用；
- 如果不存在，就在常量池中创建 `"abc"`，然后返回引用。

因此，s1 指向的是**常量池中的对象。**

```
String s2 = new String("abc");
```

这种方式会发生两步：

1. 先在字符串常量池中查找 `"abc"`，如果没有则创建；
2. 然后在堆中再创建一个新的 String 对象。

所以，s2 指向的是**堆中的新对象**，而不是常量池中的对象。

---

② 对象数量不同

```
String s1 = "abc";
```

👉 只创建 1 个对象（在字符串常量池）。

```
String s2 = new String("abc");
```

👉 通常会创建 2 个对象：

- 常量池中的 `"abc"`
- 堆中的 String 对象

---

③ 内存位置不同

变量	存储位置
s1	字符串常量池
s2	堆

---

④ 比较结果不同

```
String s1 = "abc";
String s2 = new String("abc");

System.out.println(s1 == s2); // false
System.out.println(s1.equals(s2)); // true
```

原因：

- `==` 比较的是地址；
- `equals()` 比较的是内容。

---

⑤ 性能和使用场景

- 字符串常量池具有**复用机制**，节省内存，提高性能；
- 使用 `new String()` 会创建额外对象，增加 GC 压力；
- 一般开发中推荐使用字符串字面量。

---

⑥ 加分点（高级）

可以使用 `intern()` 方法：

```
String s3 = new String("abc").intern();
System.out.println(s1 == s3); // true
```

它会返回常量池中的字符串引用。

✅ 一句话总结（面试高频）

👉 **字面量方式创建的字符串存储在常量池中，可复用；new 关键字会在堆中创建新的对象，不会复用常量池中的引用。**

引用类型有哪些？有什么区别？

引用类型主要分为强软弱虚四种：

- 强引用指的就是代码中普遍存在的赋值方式，比如A a = new A()这种。强引用关联的对象，永远不会被GC回收。
- 软引用可以用SoftReference来描述，指的是那些有用但是不是必须要的对象。系统在发生内存溢出前会对这类引用的对象进行回收。
- 弱引用可以用WeakReference来描述，他的强度比软引用更低一点，弱引用的对象下一次GC的时候一定会被回收，而不管内存是否足够。
- 虚引用也被称作幻影引用，是最弱的引用关系，可以用PhantomReference来描述，他必须和ReferenceQueue一起使用，同样的当发生GC的时候，虚引用也会被回收。可以用虚引用来管理堆外内存。

弱引用了解吗?举例说明在哪里可以用?

Java中的弱引用是一种引用类型，它不会阻止一个对象被垃圾回收

在Java中，弱引用是通过Java.lang.ref.WeakReference类实现的。弱引用的一个主要用途是创建非强制性的对象引用，这些引用可以在内存压力大时被垃圾回收器清理，从而避免内存泄露。

弱引用的使用场景：

- 缓存系统：弱引用常用于实现缓存，使用弱引用来维护缓存，可以让JVM在需要更多内存时自动清理这些缓存对象。
- 对象池：在对象池中，弱引用可以用来管理那些暂时不使用的对象。当对象不再被强引用时，它们可以被垃圾回收，释放内存。
- 避免内存泄露：当一个对象不应该被长期引用时，使用弱引用可以防止该对象被意外地保留，从而避免潜在的内存泄露

内存泄漏和内存溢出的理解?

内存泄露：内存泄漏是指程序在运行过程中不再使用的对象仍然被引用，而无法被垃圾收集器回收，从而导致可用内存逐渐减少。

内存泄露常见原因：

- 静态集合：使用静态数据结构（如HashMap或ArrayList）存储对象，且未清理。
- 事件监听：未取消对事件源的监听，导致对象持续被引用。
- 线程：未停止的线程可能持有对象引用，无法被回收。

内存溢出：内存溢出是指Java虚拟机（JVM）在申请内存时，无法找到足够的内存，最终引发OutOfMemoryError。这通常发生在堆内存不足以存放新创建的对象时。

内存溢出常见原因：

- 大量对象创建：程序中不断创建大量对象，超出JVM堆的限制。
- 持久引用：大型数据结构（如缓存、集合等）长时间持有对象引用，导致内存累积。
- 递归调用：深度递归导致栈溢出。

JVM内存结构有哪几种内存溢出的情况?

- **堆内存溢出**：当出现Java.lang.OutOfMemoryError:Java heap space异常时，就是堆内存溢出了。原因是代码中可能存在大对象分配，或者发生了内存泄露，导致在多次GC之后，还是无法找到一块足够大的内存容纳当前对象。
- **栈溢出**：如果我们写一段程序不断的进行递归调用，而且没有退出条件，就会导致不断地进行压栈。类似这种情况，JVM 实际会抛出 StackOverFlowError；当然，如果 JVM 试图去扩展栈空间的的时候失败，则会抛出 OutOfMemoryError。
- **元空间溢出**：元空间的溢出，系统会抛出Java.lang.OutOfMemoryError: Metaspace。出现这个异常的问题的原因是系统的代码非常多或引用的第三方包非常多或者通过动态代码生成类加载等方法，导致元空间的内存占用很大。
- **直接内存内存溢出**：在使用ByteBuffer中的allocateDirect()的时候会用到，很多JavaNIO(像netty)的框架中被封装为其他的方法，出现该问题时会抛出Java.lang.OutOfMemoryError: Direct buffer memory异常。

⭐ 元空间溢出（Metaspace OOM）

当 JVM 抛出 `java.lang.OutOfMemoryError: Metaspace` 时，说明元空间发生了内存溢出。

元空间主要用于存储类的元数据（如类结构、方法信息、常量池等）。
如果系统中加载的类过多，或者类无法被回收，就可能导致元空间不断膨胀，最终发生溢出。

常见原因包括：

1. **系统加载的类数量过多**
   例如大型系统、微服务或使用了大量第三方框架，导致类元数据占用空间较大。
2. **动态生成类过多**
   比如使用 CGLIB、JDK 动态代理、ASM、ByteBuddy 等技术频繁生成代理类，如果这些类没有被卸载，就会持续占用元空间。

3. **类加载器泄漏（ClassLoader Leak）**
   某些应用（如 Web 容器热部署）中，如果旧的类加载器没有被正确回收，会导致对应加载的类元数据也无法释放，从而引发元空间 OOM。

有具体的内存泄漏和内存溢出的例子么请举例及解决方案?

> 1、静态属性导致内存泄露

会导致内存泄露的一种情况就是大量使用static静态变量。在Java中，静态属性的生命周期通常伴随着应用整个生命周期（除非ClassLoader符合垃圾回收的条件）。下面来看一个具体的会导致内存泄露的实例：

> 2、 未关闭的资源

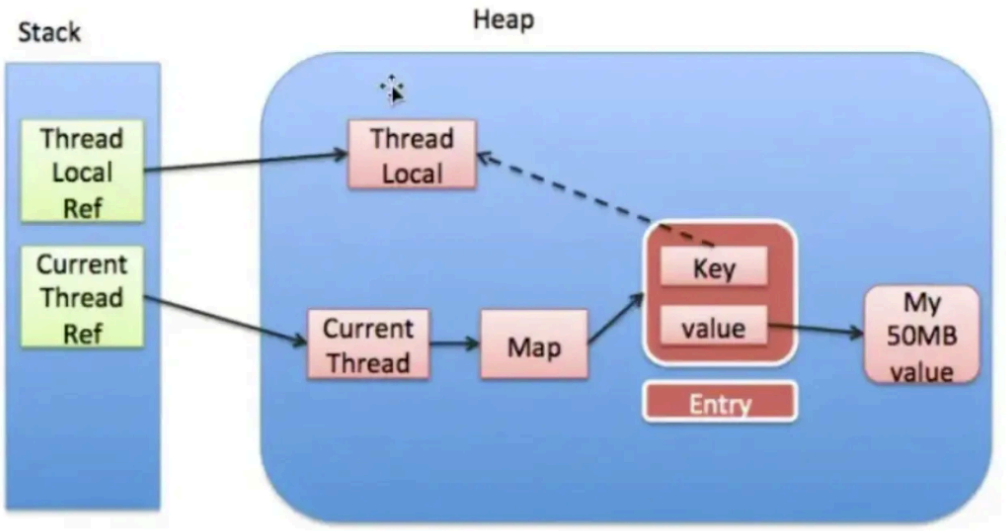
无论什么时候当我们创建一个连接或打开一个流，JVM都会分配内存给这些资源。比如，数据库链接、输入流和session对象。

忘记关闭这些资源，会阻塞内存，从而导致GC无法进行清理。特别是当程序发生异常时，没有在finally中进行资源关闭的情况。这些未正常关闭的连接，如果不进行处理，轻则影响程序性能，重则导致OutOfMemoryError异常发生。

如果进行处理呢？第一，始终记得在finally中进行资源的关闭；第二，关闭连接的自身代码不能发生异常；第三，Java7以上版本可使用try-with-resources代码方式进行资源关闭。

> 3、 使用ThreadLocal

ThreadLocal提供了线程本地变量，它可以保证访问到的变量属于当前线程，每个线程都保存有一个变量副本，每个线程的变量都不同。ThreadLocal相当于提供了一种线程隔离，将变量与线程相绑定，从而实现线程安全的特性。



ThreadLocal的实现中，每个Thread维护一个ThreadLocalMap映射表，key是ThreadLocal实例本身，value是真正需要存储的Object。

ThreadLocalMap使用ThreadLocal的弱引用作为key，如果一个ThreadLocal没有外部强引用来引用它，那么系统GC时，这个ThreadLocal势必会被回收，这样一来，ThreadLocalMap中就会出现key为null的Entry，就没有办法访问这些key为null的Entry的value。

如果当前线程迟迟不结束的话，这些key为null的Entry的value就会一直存在一条强引用链：Thread Ref -> Thread -> ThreaLocalMap -> Entry -> value永远无法回收，造成内存泄漏。

如何解决此问题?

- 第一，使用ThreadLocal提供的remove方法，可对当前线程中的value值进行移除；
- 第二，不要使用ThreadLocal.set(null) 的方式清除value，它实际上并没有清除值，而是查找与当前线程关联的Map并将键值对分别设置为当前线程和null。
- 第三，最好将ThreadLocal视为需要在finally块中关闭的资源，以确保即使在发生异常的情况下也始终关闭该资源。

垃圾回收算法哪些阶段会STW?

垃圾回收中的 STW（Stop-The-World）阶段

不同 GC 算法的 STW 情况不同，以下以常见实现为例：

标记-清除 / 标记-整理（传统全停顿）

整个 GC 过程全程 STW，包括标记和清除两个阶段。

CMS（Concurrent Mark Sweep）

阶段	是否 STW
初始标记（Initial Mark）	✅ STW（短暂）
并发标记（Concurrent Mark）	❌ 并发
重新标记（Remark）	✅ STW（较短）
并发清除（Concurrent Sweep）	❌ 并发

G1（Garbage First）

阶段	是否 STW
初始标记（Initial Mark）	✅ STW

阶段	是否 STW
并发标记（Concurrent Mark）	❌ 并发
最终标记（Final Mark / Remark）	✅ STW
筛选回收（Evacuation）	✅ STW

ZGC / Shenandoah

设计目标是极低 STW，大部分阶段并发执行：

阶段	是否 STW
初始标记根集合	✅ STW（极短，毫秒级）
并发标记	❌ 并发
再标记 / 根修复	✅ STW（极短）
并发转移/压缩	❌ 并发

总结规律

几乎所有 GC 算法都无法避免以下两类 STW：

1. **初始标记根对象**：需要一致的根集合快照，必须 STW
2. **重新标记（Remark）**：修正并发标记期间的漏标，必须 STW

优化方向就是让这两次 STW 尽可能短，其余阶段与应用线程并发执行。

什么情况使用CMS、G1?

CMS：

如果需要一个老年代的低延迟的垃圾回收器，可以使用CMS

G1：

G1垃圾回收器适用于 对内存碎片比较敏感、需要比较平衡的性能且大内存堆的场景

G1在提供较低停顿时间的同时，也保持了相对较高的吞吐量。

G1垃圾回收器的特色?

特点:

- G1最大的特点是引入分区的思路，弱化了分代的概念。
- 合理利用垃圾收集各个周期的资源，解决了其他收集器、甚至 CMS 的众多缺陷

G1 相比较 CMS 的改进：

- **算法**： G1 基于标记--整理算法, 不会产生空间碎片，在分配大对象时，不会因无法得到连续的空间，而提前触发一次 FULL GC 。
- **停顿时间可控**： G1可以通过设置预期停顿时间（Pause Time）来控制垃圾收集时间避免应用雪崩现象。
- **并行与并发**：G1 能更充分的利用 CPU 多核环境下的硬件优势，来缩短 stop the world 的停顿时间。

GC只会对堆进行GC吗?

JVM 的垃圾回收器不仅仅会对堆进行垃圾回收，它还会对方法区进行垃圾回收。